

— PROFILE

Frontend Engineer

Enterprise SaaS · AI Productivity · Mobile Platform 도메인에서
실제 제품을 개발하며 사용자 경험과 품질을 개선해 온 3+ years 경력 개발자.

DOMAINS

Enterprise Collaboration · AI Productivity · React Native Mobile

SPECIALTIES

권한 분기 UI · AI 응답 스트리밍 · WebView ↔ Native Bridge

CORE STACK

React · React Native · TypeScript · Redux Toolkit · Context API

COLLABORATION

기획 · 디자인 · Native · Backend · AI 팀과의 협업 경험

CAREER FOCUS

제품 품질 · 유지보수성 · 사용자 경험을 함께 책임지는 Frontend 엔지니어링

PERSONAL PRODUCTS

WhatFlix (Movie Community) · Pixel Garden Portfolio

— SUMMARY

경력 요약 · 한눈에 보는 업무 범위

B2B Enterprise SaaS · AI 생산성 · 모바일 — 세 도메인에서 **화면 설계와 구현, 공통 컴포넌트 추출, AI API 연동, Web ↔ Native Bridge** 등 서비스 전체를 가로 지르는 영역의 Frontend 엔지니어링을 담당. 기술을 먼저 고르지 않고, 사용자 문제와 운영 비용을 함께 고려한 구조 설계를 우선합니다.

— PROJECT HIGHLIGHTS

NO.	PROJECT	담당 업무 요약	STACK
01	Enterprise Collaboration Platform B2B 협업 SaaS · 대기업/중견기업 임직원 대상	문서 공유 · 조직도 트리 · 역할별 권한 관리 UI · 댓글 · 공통 선택 컴포넌트 추출 · 권한 판단 Custom Hook 진입점 설계.	React · TypeScript · Redux Toolkit · REST · WebSocket
02	Enterprise AI Productivity Platform 기업 AI 생산성 SaaS · Web + Mobile	AI Chat 스트리밍 UI · OCR / STT 화면 흐름 · Prompt Library · AI 회의록 뷰어 · AI 기능 공통 진행/실패 상태 이름 정리.	React · React Native · Context API · Native Module · AI API
03	Enterprise Mobile Platform React Native 기반 기업 모바일 앱	WebView ↔ Native Bridge 메시지 프로토콜 · Camera · Gallery · 파일 접근 · Deep Link · 권한 요청 흐름 · 플랫폼 분기 공용 유틸.	React Native · TypeScript · WebView Bridge · Native Module
04	Personal Products WhatFlix · Pixel Garden Portfolio	기획 · 디자인 · 개발 · 운영을 직접 소유한 개인 제품. 외부 API · AI 추천 · 커뮤니티 · 인터랙티브 UI 설계 경험.	React · TypeScript · Spring Boot · Pixel UI

Enterprise Collaboration Platform

문서 공유 · 권한 관리 · 조직도 · 댓글 · 메신저 · 화상회의를 단일 플랫폼으로 통합한 B2B 협업 SaaS.

서비스 대상	담당 역할	팀 구성	사용 기술
대기업·중견기업 임직원 (B2B Enterprise SaaS)	Frontend Engineer 협업 기능 영역 단독 개발	기획 · 디자인 · FE · BE 전사 운영 단계 서비스	React · TypeScript Redux Toolkit · WebSocket

— 담당 화면

화면 6개

문서 공유 · 오픈 링크 권한 완료 PW 링크 발급 F-01 · 링크 발급	조직도 (Tree) 본부 팀 A - 사원 ① - 사원 ② 팀 B F-02 · 조직도	역할별 권한 관리 Admin Editor Viewer Guest F-03 · 권한 관리	댓글 · 다중 선택 @멘션 · 다중 선택 F-04 · 댓글	메신저 UI 실시간 (WebSocket) F-05 · 메신저	화상회의 UI Active speaker F-06 · 화상회의
--	---	--	--	---	--

— 담당 기능

화면 · API 연동 · 공통 컴포넌트 · 상태 관리

✓ 문서 공유 · 오픈 링크 발급 완료일 · 권한 · 비밀번호 설정 UI, 링크 발급 API 연동, 대상자 검색 인터랙션을 담당.	✓ 조직도 트리 뷰어 · 선택 UI 펼침 · 접힘 상태 관리, 검색 하이라이트, 대용량 조직에서도 스크롤이 끊어지지 않도록 렌더링 방식을 조정.	✓ 역할별 권한 관리 UI 버튼 · 메뉴 · 데이터 단위로 분기되던 권한 판단을 Custom Hook 진입점으로 모아 정리.
✓ 댓글 · 다중 선택 인터랙션 작성 · 수정 · 삭제 · 멘션, 다중 선택 일괄 작업, 조직도 검색과 연동된 멘션 UI를 담당.	✓ 공통 선택 컴포넌트 (Picker) 여러 화면에 중복 구현되던 사용자 · 부서 · 문서 선택 UI를 단일 컴포넌트로 추출 · 재사용.	✓ 실시간 UI · 메신저 / 화상회의 WebSocket 알림, 신규 메시지 표시, 화상회의 참여자 상태 동기화를 담당.

권한 정책이 자주 바뀌면서, 같은 코드를 여러 화면에서 반복 수정하고 있었다.

운영 단계에서 권한 정책이 반복적으로 바뀌었고, 그때마다 화면마다 권한 분기 코드를 따로 손보는 상황이 누적되며 같은 버그가 다른 화면에서 다시 나타나는 일이 생겼습니다.

상황

운영 중인 협업 SaaS에서는 새 기능 요청보다 **권한 정책 변경**이 더 자주 발생했습니다. "관리자만 가능"이 "팀장 이상"으로, 다시 "프로젝트 소유자"로 바뀌는 식의 변경이 잦았고, 그때마다 같은 정책이 들어가는 화면 여러 곳을 동시에 수정해야 했습니다.

사용자 입장에서 보이던 모습

- 권한 변경 후 일부 화면에서는 메뉴가 사라졌지만, 다른 화면에서는 같은 버튼이 그대로 노출.
- 이미 열려 있던 화면에서는 변경 전 권한 상태가 남아 있어, 클릭 시 권한 오류가 발생.
- 관리자가 권한을 회수한 직후에도, 사용자가 화면을 새로고침하기 전까지는 이전 권한이 유지되는 것처럼 보임.

원인 - 권한 판단이 화면마다 흩어져 있었다

권한을 어디서 판단할지에 대한 약속이 없었고, 화면마다 직접 사용자 정보와 역할을 비교해 분기하고 있었습니다.

- 화면마다 같은 정책이 따로 들어가 있어, 정책이 바뀌면 같은 수정을 여러 번 반복.
- 실시간 권한 변경 이벤트도 화면마다 따로 처리해 일부 화면은 갱신되지 않음.
- 새 합류한 개발자가 "이 버튼이 왜 비활성인지" 추적할 단일 위치가 없었음.

디버깅

- QA에서 권한 회수 시나리오를 두 갈래로 나눠 재현 — "새로 진입한 화면" vs "이미 열려 있던 화면".
- 권한을 판단하는 위치를 코드에서 모두 찾아보니, 비슷한 분기 로직이 여러 화면에 분산되어 있었음.
- 실시간 권한 변경 메시지가 정상 수신되는데도 일부 화면에 반영되지 않는 이유 — 화면마다 별도 상태를 들고 있었기 때문.

내가 한 일

권한 판단을 **하나의 진입점**에서만 하도록 정리했습니다. 화면은 사용자 정보 대신 "이 기능을 쓸 수 있는지"를 묻고, 권한 변경 이벤트도 한 곳에서 받아 화면이 함께 갱신되도록 바꿨습니다.

한계 / 남은 일

모든 화면을 한 번에 정리하지는 못했습니다. 새 기능부터 적용하고, 기존 화면은 권한 정책이 바뀔 때마다 같이 옮기는 식으로 점진적으로 이동했습니다.

배운 점

운영에서는 새 기능보다 **정책 변경**이 더 자주 발생합니다. 이후에는 화면을 그리기 전에 권한 기준이 어디에 있어야 하는지를 먼저 정리하고 개발하게 됐습니다.

권한 판단을 화면 바깥으로 옮기기로 했다.

여러 방식을 비교했고, 화면 코드에서 직접 사용자/역할을 비교하는 대신 "이 기능을 쓸 수 있는지"만 묻도록 진입점을 한 곳으로 모았습니다.

풀어야 했던 문제

같은 권한 정책이 여러 화면에 반복 구현되어 정책 변경 시 같은 수정을 반복했고, 화면마다 따로 보유한 권한 상태가 어긋나며 같은 사용자에게 화면별로 다른 권한이 보이는 상황이 생겼습니다.

만족시켜야 했던 조건

- 권한 판단의 위치가 하나여서, 정책이 바뀌면 한 곳만 보면 되는 구조.
- "읽기 가능, 댓글 불가"처럼 한 리소스에 여러 권한이 섞이는 표현이 가능할 것.
- 권한 변경 이벤트가 들어왔을 때 화면들이 함께 갱신될 것.
- 새 합류한 동료가 "이 버튼이 왜 비활성인지"를 한 번에 찾을 수 있을 것.
- 백엔드 권한 모델이 바뀌어도, 화면 측 코드는 같은 질문(쓸 수 있는지/없는지)만 유지될 것.

비교한 선택지

방식	장점	걸렸던 점	선택
서버에서만 판단	UI 코드가 단순	메뉴 노출 깜빡임, 진입 후 권한 오류	✗
HOC로 감싸기	선언적, 컴포넌트 단위	래퍼 중첩, 부분 권한 표현이 부자연	✗
Context Provider 공유	트리 전반에 같은 권한 노출	권한이 자주 바뀌면 광범위 리렌더	✗
화면에서 store 직접 접근	기존 store 그대로 활용	호출부가 분산되어 정책 변경 비용 큼	✗
Custom Hook 진입점 화면은 "쓸 수 있는지"만 묻는다	호출 위치가 명시적, 부분 권한 표현 자유, 구독 단위로만 갱신	Hook 인터페이스 합의 비용 필요	✓

결정

권한 판단은 **Custom Hook 한 진입점**에 두기로 했습니다. 화면은 capability(쓸 수 있는지)만 묻고, 변경 이벤트도 같은 곳에서 받아 처리합니다.

트레이드오프

한 번 정한 인터페이스가 굳기 전까지 팀 합의 비용이 있었고, 한꺼번에 모든 화면을 옮기지 못했습니다. 새 화면부터 적용하고 기존 화면은 점진적으로 이동하기로 합의했습니다.

결과

권한 정책이 바뀌었을 때 수정 범위를 Hook 중심으로 줄였습니다. 새 합류 동료가 "권한이 어디서 정해지나"를 한 곳에서 확인할 수 있게 됐고, 같은 종류의 회귀 이슈가 다시 발생하지는 않았습니다.

배운 점

어떤 도구를 쓰느냐보다 **판단을 어디에 두느냐**가 먼저였습니다. 다시 한다면 Hook 인터페이스의 합의를 좀 더 일찍, 더 구체적인 예시와 함께 시작하겠습니다.

Enterprise AI Productivity Platform

반복 업무를 AI 기능으로 대체해 임직원 생산성을 높이는 통합 AI SaaS — **AI API를 안정적인 사용자 경험으로 연결**하는 Frontend 영역 담당.

서비스 대상	담당 역할	팀 구성	사용 기술
기업 임직원	FE · React Native Engineer	기획 · AI · FE · Native · BE	React · React Native
Web · Mobile 양쪽	AI 기능 UI 통합 / Native 연동	운영 단계 서비스	Context API · AI API · Native Module

- 담당 화면화면 6개 · AI 기능 UI 통합

AI Chat · 스트리밍 질문 ... ● 스트리밍 AI 응답 F-01 · AI 채팅	OCR (Camera → Text) capture Native SDK F-02 · OCR	STT (Voice → Text) 녹음 중... F-03 · STT	Prompt Library 즐거찾기 · 카테고리 F-04 · 프롬프트	AI 회의록 분석 ▶ 요약 ▶ 액션 F-05 · 회의록	중요문서함 개인 보관함 분리 F-06 · 중요문서함
--	---	--	---	--	---

- 담당 기능AI 기능 UI 통합 · Native 연동 · 상태 관리 · UX 설계

✓ AI Chat · 스트리밍 UI AI API의 스트리밍 응답을 화면에 순차적으로 보여주고, 응답 진행 중에도 다른 UI가 흐름에 영향받지 않도록 담당.	✓ OCR (Camera → Text) Native SDK와 AI API를 따라 이동하는 화면 흐름을 구성하고, 진행 상태와 실패 안내 UI를 담당.	✓ STT (Voice → Text) 녹음 · 변환 · 편집 화면을 시리즈적으로 연결하고, Native 권한 요청 흐름을 함께 담당.
✓ Prompt Library · 즐겨찾기 자주 쓰는 프롬프트를 저장 · 분류 · 검색 · 호출하는 흐름을 담당.	✓ AI 회의록 · 중요문서함 긴 AI 응답에서 요약·액션 항목을 분리해 보여주고, 사용자가 보관·관리할 수 있는 UX를 담당.	✓ AI 기능 공통 상태 흐름 AI Chat · OCR · STT가 공유하는 진행·실패 상태 이름을 정리하고, 화면 이탈 시 응답 수신을 끝내는 흐름을 담당.

AI 응답이 길어질수록 화면이 느려졌고, 실패 시 다음에 무엇을 할지 알기 어려웠다.

운영 중 AI 응답 지연 상황과 실패 상황 모두에서 사용자가 현재 진행 상태를 이해하기 어렵다는 피드백이 반복되었고, AI 기능별로 처리 방식도 제각각이라 동일한 종류의 버그가 화면별로 다시 나타났습니다.

상황

AI Chat의 긴 응답이 흘러나오는 동안 화면의 다른 입력이 느리게 반영되거나, 다른 화면으로 이동했다 돌아오면 응답이 어디까지 진행됐는지 알기 어렵다는 피드백이 반복됐습니다. OCR · STT의 실패 화면도 단순한 "다시 시도" 버튼만 있어 사용자가 다음에 무엇을 할 수 있는지 안내가 부족했습니다.

사용자 입장에서 보이던 모습

- 긴 응답 도중 입력창 타이핑이 한 박자 늦게 반영되는 느낌.
- 화면을 이동했다 돌아오면 답변 진행 상태가 모호하거나 일부만 남아 있는 경우.
- OCR / STT 실패 시 어디서 실패했는지, 다시 시도해야 하는지, 그냥 메모하면 되는지 안내가 없음.

원인

- 응답을 받는 단위와 화면을 갱신하는 단위가 같았다 — 토큰이 들어올 때마다 화면을 다시 그리면서 다른 영역에도 영향을 줌.
- 화면을 이탈해도 응답 수신은 끊지 않아서, 돌아왔을 때 결과의 일관성이 흔들림.
- AI Chat · OCR · STT가 각자 다른 방식으로 진행/실패 상태를 표현해 같은 종류의 문제가 기능마다 다시 발생.

디버깅

- 입력 지연이 어디서 시작되는지 확인하기 위해, 메시지 영역 갱신 빈도와 다른 UI의 반응성을 함께 관찰.
- 화면 이탈 시 응답 수신이 끝나는지 확인 — 정리되지 않고 남는 경우가 있었음.
- AI 기능별로 실패 처리 코드를 비교해 보니, 같은 의미의 상태를 다른 이름으로 다루고 있었음.

내가 한 일

응답을 받는 단위와 화면 갱신 단위를 분리해, 사용자에게는 "흐르는 느낌"을 남기되 다른 UI에 영향을 덜 주도록 했습니다. 화면 이탈 시 응답 수신을 정리하는 흐름을 한 곳에서 처리하도록 모았고, AI 기능 전체의 진행/실패 상태를 같은 이름으로 정리했습니다.

한계 / 남은 일

저사양 단말과 네트워크가 불안정한 환경까지 충분히 검증하지는 못했습니다. 같은 구조 위에서 환경별 보정값을 더 정밀하게 만드는 일이 남아 있다고 봅니다.

배운 점

AI 기능에서는 응답을 보여주는 일보다 **기다림과 실패를 다루는 일**이 더 큰 비중을 차지했습니다. "성공 화면"보다 먼저 "기다림 화면 · 실패 화면"을 설계하는 습관이 생겼습니다.

"받는 단위"와 "보여주는 단위"를 분리하기로 했다.

AI 응답이 흘러가는 체감은 유지하면서도 다른 UI에 영향을 덜 주도록, 응답을 받는 단위와 화면을 갱신하는 단위를 분리했고, AI 기능 전체의 진행/실패 상태를 같은 이름으로 정리했습니다.

풀어야 했던 문제

응답이 길어질 때 다른 UI까지 느려지는 문제, 화면을 떠났을 때 응답이 정리되지 않는 문제, 그리고 AI 기능마다 진행/실패 상태를 다르게 다뤄 같은 종류의 문제가 반복되는 문제를 함께 풀어야 했습니다.

만족시켜야 했던 조건

- 응답이 흘러가는 체감은 유지할 것 — 다 모은 뒤 한 번에 보여주는 방식은 제외.
- 응답이 길어져도 다른 UI(입력창 등)에 영향이 가지 않을 것.
- 화면 이탈 / 새 질문 / 명시 취소가 같은 흐름으로 응답 수신을 정리할 것.
- 실패가 "다시 시도" 버튼 하나로 끝나지 않고, 사용자가 다음 행동을 알 수 있을 것.
- Web과 모바일이 같은 상태 이름을 공유해서, 한쪽에서 고친 것이 다른 쪽에서 다시 발생하지 않게 할 것.

비교한 선택지

방식	장점	걸렸던 점	선택
갱신 빈도를 낮추기	코드 변경 최소화	흐름이 끊겨 "AI가 멈춘 것 같다"는 피드백	✗
응답 전부 모아 한 번에 표시	렌더 비용은 가장 적음	"흐르는" 체감 사라짐 — 제품 핵심과 충돌	✗
화면 컴포넌트만 메모이즈	국소 효과	상태 분리·정리 누락이라는 근본 원인 해결 못 함	✗
받는 단위 / 보여주는 단위 분리 + AI 기능 공통 상태 흐름	"흐르는" 체감 유지 + 다른 UI 보호 + 단일 정리 지점	초기 구조 설계와 팀 약속 비용 필요	✓

결정

응답을 받는 단위와 화면 갱신 단위를 분리. AI 기능 전체가 같은 진행/실패 이름을 공유하고, 화면 이탈 시 응답 수신을 정리하는 흐름을 한 곳에서 다룹니다.

트레이드오프

"흐름감"과 "안정성" 사이에 절충값을 정해야 했고, 디자인 · 기획과 함께 체감 테스트 후 합의했습니다. 환경에 따라 같은 값이 다르게 느껴지는 경우가 있다는 점은 한계로 남았습니다.

결과

긴 응답에서도 다른 UI의 반응성이 같이 흔들리는 현상이 줄었습니다. 같은 진행/실패 이름을 공유하면서, 새 AI 기능을 붙일 때 화면별로 다시 짤 일이 줄어들었습니다.

배운 점

AI 기능을 만든다는 건 "성공 화면"을 그리는 일이 아니라, **기다림 · 실패 · 취소까지 한 사용자 흐름으로 다루는 일**이라는 점을 배웠습니다.

Enterprise Mobile Platform

PC에서만 쓰던 협업 · AI 서비스를 모바일에서도 같은 품질로 사용할 수 있도록, **WebView와 Native 기능을 연결하는 Bridge 레이어**를 다룬 React Native 기반 기업 모바일 앱.

— 담당 화면

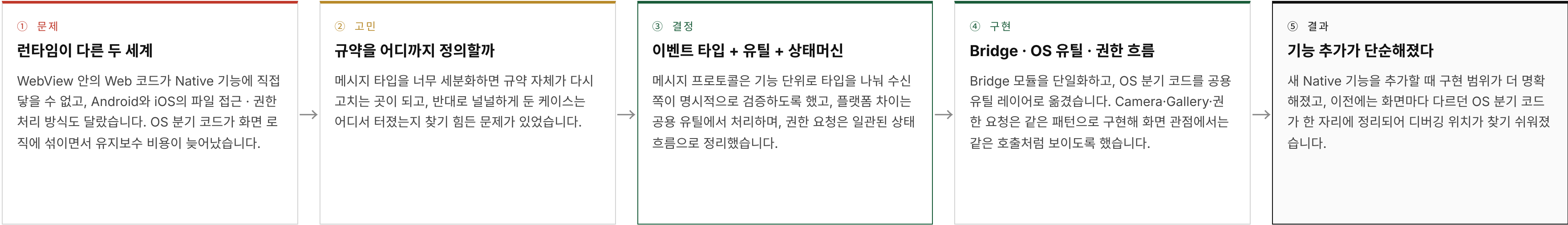


화면 5개

— 담당 기능

- ✓ **WebView ↔ Native Bridge**
WebView와 Native 간 메시지 타입을 명시적으로 정의하고, 양쪽이 같은 규약에서 통신하도록 구성.
- ✓ **Native 기능 연동**
Camera · Gallery · 파일 접근 · Native Module 호출 흐름을 화면 관점에서 구성.
- ✓ **Permission 상태머신**
권한 요청 → 거부 → 설정 이동 → 재요청으로 이어지는 상태 흐름을 설계·구현.
- ✓ **Deep Link · 플랫폼 분기 유틸**
Android와 iOS의 파일 접근 방식 차이를 화면 코드가 직접 다루지 않도록 공용 유틸 레이어에 분리.

— 문제 → 고민 → 결정 → 구현 → 결과



— 배운 점

크로스플랫폼 작업에서는 OS 차이를 화면 코드가 직접 다루지 않도록 **경계 레이어의 위치를 먼저 정하는 것**이 이후 기능을 추가할 때의 비용을 줄여준다는 점을 배웠습니다.

STACK

React Native

TS

Bridge

Native Module

RNFS

– PERSONAL PRODUCTS

기획부터 운영까지 직접 소유한 개인 제품

WHATFLIX

ROLE

Movie Community

기획 · 개발 · 운영

"오늘 뭐 볼지 모르겠다"는 일상의 불편에서 출발한 개인 제품. 외부 영화 데이터 API · AI 추천 · 커뮤니티 기능을 직접 기획해 결합.

PROBLEM

기존 OTT 추천은 짧은 시청 기록에 강하게 묶여, "내가 왜 좋아했는지"가 다음 추천으로 이어지지 않음.

SOLUTION

외부 영화 API로 메타데이터를 받고, 사용자 코멘트 · 평점을 추천 입력으로 함께 사용. **"사람의 언어가 섞인 추천"**이 핵심 가설.

RESULT

기획 · API 연동 · AI 추천 · 커뮤니티 화면을 하나의 일관된 흐름으로 구축했고, 개인 운영을 통해 실제 사용자 피드백을 받아 반영해봤습니다.

LEARNED

소유자 입장에서서는 기능을 늘리는 것보다 **기능을 줄이는 결정이 더 어렵다**는 것을 체감했습니다.

React

TypeScript

Spring Boot

External Movie API

AI Recommendation

PIXEL GARDEN PORTFOLIO

ROLE

Interactive Portfolio

기획 · 디자인 · 개발

단순 이력 페이지가 아니라, **게임형 UI와 인터랙션**을 활용해 개발 경험을 하나의 제품처럼 전달하는 인터랙티브 포트폴리오.

PROBLEM

일반적인 포트폴리오는 텍스트와 스크린샷의 나열에 그쳐, 개발자의 성향과 인터랙션 감각이 잘 전달되지 않음.

SOLUTION

픽셀 그래픽과 게임형 인터랙션을 차용해 **"읽는" 포트폴리오**가 아닌 **"움직여 보는" 포트폴리오**를 기획 · 구현.

RESULT

방문자가 페이지를 단순 스크롤하는 것이 아니라, 직접 조작하며 콘텐츠를 발견하는 흐름의 사용자 경험을 만들었습니다.

LEARNED

전달 매체의 형식 자체가 전달되는 내용을 바꿀 수 있다는 것을 배웠습니다.

React

TypeScript

Pixel UI

Game-style Interaction

— EXPERIENCE SUMMARY

이런 경험을 가지고 있습니다.

01 Enterprise SaaS 협업 플랫폼 개발 경험

문서 공유 · 조직도 · 역할별 권한 · 댓글 · 메신저 · 화상회의 영역을 단독 개발했습니다. 운영 중에는 반복되는 권한 정책 변경 속에서 유지보수성을 높이기 위한 구조 조정을 진행했습니다.

02 AI 기능(AI Chat · OCR · STT) UI 통합 경험

AI API를 화면에 연결·통합하면서, 응답 지연·실패·취소 상황에서도 사용자가 현재 진행 상태를 이해할 수 있도록 UI를 개선했습니다.

03 React Native 기반 모바일 서비스 개발 경험

WebView Bridge · Native Module · Camera · 권한 요청 흐름 · Deep Link · 플랫폼 분기를 직접 구현했고, OS 차이를 화면 외부의 공용 유틸 레이어에서 다루도록 정리했습니다.

04 사용자 경험과 유지보수성을 고려한 구조 설계 경험

Custom Hook · 공통 컴포넌트 · 메시지 프로토콜처럼, 같은 패턴이 다른 맥락에서 반복되는 것을 확인한 뒤 추출하는 식으로 조기 추상화의 부채를 피해 왔습니다.

05 다양한 직군과 협업하며 제품을 개선한 경험

기획 · 디자인 · AI · Native · Backend와 메시지 타입, 권한 모델 같은 경계 규약을 먼저 합의하고, 그 위에서 운영 단계의 제품을 함께 개선했습니다.

— IN SUMMARY

Problem first.
Screen next.
Technology last.

3 DOMAINS

Enterprise Collaboration
AI Productivity (Web · Mobile)
React Native Mobile

PERSONAL PRODUCTS

WhatFlix · Pixel Garden Portfolio

읽어 주셔서 감사합니다. 더 자세한 화면 설계 과정과 의사결정 트레이드 오프는 인터뷰에서 이어서 나누고 싶습니다.